

C++

Les principaux points traités sont:

- ❑ Les structures de base du langage C++.
- ❑ L'allocation dynamique et la maîtrise du fonctionnement des pointeurs.
- ❑ Le concept de classes et d'objets, les membres, fonctions membres, fonctions amies et le cas particulier très important des constructeurs et du destructeur.
- ❑ La notion d'héritage, simple puis multiple avec les notions de polymorphisme.
- ❑ La gestion des exceptions.

CHAPITRE 1

PRINCIPES DE BASE DU

LANGAGE C++

Histoire du C/C++

- ❑ Le C a été inventé au cours de l'année 1972 dans les laboratoires Bell par Dennis Ritchie et Ken Thompson.
- ❑ En 1978, Brian Kernighan, qui aida à populariser le C, publia le livre « The C programming Language », le K&R, qui décrit le C « traditionnel » ou C ANSI.
- ❑ Dans les années 80, Bjarne Stroustrup développa le C++ afin d'améliorer le C, en lui ajoutant des « classes ». Le premier nom de ce langage fut d'ailleurs « C with classes ».
- ❑ Ce fut en 1998 que le C++ fut normalisé pour la première fois. Une autre norme corrigée fut adoptée en 2003.
- ❑ Une mise à jour importante fut C++11, C++14, C++17, ajoutant de nombreuses fonctionnalités au langage.
- ❑ Toutes ces normes permettent une écriture indépendante du compilateur. Le C++ est le même partout, pourvu qu'on respecte ces normes.

Aspect impératif du C++

- ❑ Le C++ est une surcouche de C, avec quelques incompatibilités de syntaxe
- ❑ Un programme C est la plupart du temps un programme C++.
- ❑ Donc on peut faire du « C+ » c'est-à-dire du C++ sans objet.
- ❑ Impératif : les instructions se suivent dans un ordre précis et transmis au processeur de la machine dans cet ordre.
- ❑ Impératif et objet ne se contredisent pas, C++ est un langage multi paradigmes. Il respecte à la fois le paradigme objet et impératif.
- ❑ On va donc commencer par faire du C++ impératif.
- ❑ L'approche procédurale ou impérative consiste à découper un programme en un ensemble de fonctions (ou procédures). Ces fonctions contiennent un certain nombre d'instructions qui ont pour but de réaliser un traitement particulier.

Programmation

- ❑ La programmation consiste à écrire des séquences d'instructions dans un langage compréhensible par les humains, tel que le C++, qui seront ensuite traduites en instructions élémentaires directement exécutables par le microprocesseur de l'ordinateur.
- ❑ C'est un programme appelé « compilateur » qui a pour charge de traduire un programme rédigé en C++, en une séquence d'instructions exécutable par le microprocesseur.

Étapes de la programmation

- ❑ Pour un problème donné, il faut d'abord concevoir la séquence d'instructions que le programme exécutera.
- ❑ Ensuite, on écrit le programme C++ qui correspond à cette séquence d'instructions et on le compile pour créer un programme exécutable.
- ❑ La compilation peut échouer, parce que le programme ne respecte pas les règles du C++ ; pour que le programme soit compilé, il faut qu'il respecte les règles du C++.
- ❑ Enfin, quand le programme est compilé, on le teste en l'exécutant. Il se peut que le programme ne fasse pas ce que l'on souhaite parce que l'on a mal conçu la séquence d'instructions au départ. Dans ce cas, il faut repenser la séquence d'instructions, modifier le programme, et continuer ce cycle de développement.

ÉCRITURE DU PREMIER PROGRAMME

❑ On commence par ouvrir un environnement de développement. D'usage, quand on apprend un nouveau langage, on commence avec un programme qui affiche le message « Hello world ». En C++, il s'écrit comme ceci :

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "Hello world!" << endl;
```

```
    return 0 ;
```

```
}
```

Structure générale d'un programme

Fonction main

- ❑ Tout programme doit avoir un point d'entrée nommé main

```
int main() {  
    return 0;  
}
```

- ❑ La fonction main est la fonction appelée par le système d'exploitation lors de l'exécution du programme
 - ❖ { et } délimitent le corps de la fonction
 - ❖ main retourne un entier au système: 0 (zéro) veut dire succès
 - ❖ Chaque expression doit finir par; (point virgule)

Structure générale d'un programme

Commentaires

❑ En C++: Commentaires sur plusieurs lignes : délimités par /* (début) et */ (fin).

/* Un commentaire en une seule ligne */

/*

*** Un commentaire sur plusieurs**

*** lignes**

***/**

❑ Commentaires sur une seule ligne : délimités par // (début) et fin de ligne

// Un commentaire jusqu'à la fin de cette ligne

Structure générale d'un programme

Fichiers Sources

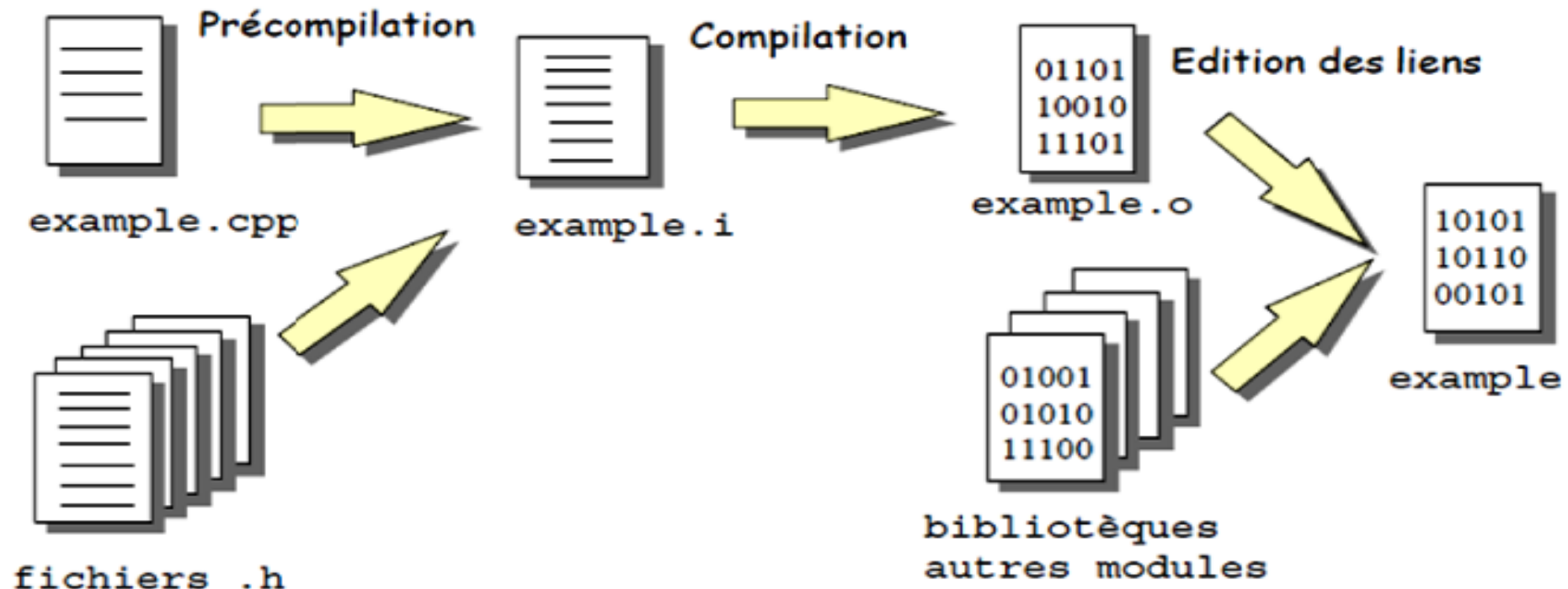
- ❑ Un programme est généralement constitué de plusieurs modules, chaque module est composé de deux fichiers sources :
 - ❖ Un fichier contenant la description de l'interface du module
 - ❖ Un fichier contenant l'implémentation proprement dite du module
- ❑ Un suffixe est utilisé pour déterminer le type de fichier
 - ❖ **.h, .H, .hpp, .hxx** : pour les fichiers de description d'interface (header files ou include files)
 - ❖ **.c, .cc, .cxx, .cpp, .c++** : pour les fichiers d'implémentation

Structure générale d'un programme

Fichiers Sources

Dans un fichier source on peut trouver:

- ☐ Commentaires
- ☐ Instructions pré-processeur
- ☐ Instructions C++



Structure générale d'un programme

Bibliothèque de C++

- ❑ C++ possède une bibliothèque très riche, qui comporte un très grand nombre d'outils (fonctions, types, ...) qui permettent de faciliter la programmation.
- ❑ Elle intègre en plus de la bibliothèque standard de C, des librairies de gestion des entrées-sorties ainsi que des outils de manipulation des chaînes de caractères, des tableaux et d'autres structures de données.
- ❑ Pour utiliser, en C++, les outils qui existaient dans la bibliothèque standard de C (stdio.h, string.h, ...) ainsi que certains nouveaux outils, il suffit de spécifier avec la directive include le fichier entête (.h) souhaité.

Structure générale d'un programme

Bibliothèque de C++

```
#include <iostream> // En C++ :: « Flux d'entrée-sortie »
```

```
#include <cstdio> // En C : « Flux d'entrée-sortie »
```

□Exemple:

```
#include <iostream>
```

```
#include <cstdio> // Les librairie C
```

```
int main(){
```

```
    std ::cout << "Hello !" << std ::endl;
```

```
    printf(" Hello bis !\n ") ; // Les instructions C
```

```
    return 0;
```

```
}
```

Structure générale d'un programme

Espace de noms en C++

- ❑ Pour des raisons liées à la POO (généricité, modularité...) et pour éviter certains conflits qui peuvent surgir entre les différents noms des outils utilisés (prédéfinis ou définis par l'utilisateur),
- ❑ C++ introduit la notion de **namespace** (espace de noms), ce qui permet de définir des zones de déclaration et de définitions des différents outils (variables, fonctions, types,...).
- ❑ Ainsi, chaque élément défini dans un programme ou dans une bibliothèque appartient désormais à un **namespace**.
- ❑ La plupart des outils d'Entrées/Sorties de la bibliothèque de C++ appartiennent à un namespace nommé **"std"**.

Structure générale d'un programme

❑ Syntaxe : **using namespace std;**

❑ Exemple d'utilisation:

```
#include <iostream>
```

```
#include <conio.h>
```

```
using namespace std; // Importation de l'espace de nom
```

```
void main(){
```

```
    double x, y;
```

```
    cout << "X:"; // Le préfixe n'est plus requis :
```

```
    cin >> x;
```

```
    cout << "Y:"; // Il est toujours possible d'utiliser le préfixe : std::cin >> y;
```

```
    cout << "x * y = " << x * y << endl;
```

```
    _getch(); // Les fonctions de la bibliothèque C sont toujours utilisables
```

```
}
```

Structure générale d'un programme

Variables

- ❑ Une variable est un moyen en programmation de stocker une donnée, d'y associer un nom.
- ❑ Concrètement, dans un programme s'exécutant sur une machine, une variable correspond à une zone en mémoire, qui a un nom que l'on appelle un identificateur de la variable.
- ❑ Cette zone mémoire stocke une valeur, associée à l'identificateur. C++ exige également que l'on définisse le genre associé à chaque donnée, ce que l'on appelle le type de la variable. Il faut le spécifier au moment de la déclaration.

Structure générale d'un programme

Déclaration des variables

❑ La déclaration d'une variable se fait comme suit :

type identificateur;

❑ Initialiser une variable consiste à lui donner une valeur de départ. Ceci se fait habituellement au moment de la déclaration. :

type identificateur(valeur_initiale);

❑ En C++, il est possible de déclarer une variable sans l'initialiser. Cependant, il faut toujours initialiser une variable avant d'utiliser sa valeur. En effet, les traitements avec une variable non initialisée sont complètement imprédictibles, et peuvent produire des résultats erronés. **Il est alors déconseillé de déclarer une variable sans initialisation.**

Structure générale d'un programme

Déclaration des variables

❑ Exemple

```
int m(1);
```

```
int p(1), q(0);
```

```
double x(0.1), y;
```

- ❑ Le type associé à la variable au moment de sa déclaration est fondamental. Il conditionne le type de traitement que l'on peut réaliser avec la variable en question. Il faut être attentif au fait que le type d'une variable ne peut pas changer : lorsque l'on déclare une variable d'un certain type, cette variable garde ce type jusqu'à la fin de son existence dans le programme.

Structure générale d'un programme

Nom de variables

- ❑ Il existe un certain nombre de conventions à respecter lorsque l'on déclare une variable
- ❑ L'identificateur doit être constitué de lettres et de chiffres sans espace ni symboles
- ❑ Le caractère souligné _ est considéré comme une lettre
- ❑ Les accents ne sont pas autorisés
- ❑ Le premier caractère est nécessairement une lettre
- ❑ L'identificateur ne doit pas être un mot réservé du langage (par exemple «if»)
- ❑ Les majuscules et minuscules sont autorisées mais pas équivalentes ; les noms **ligne** et **Ligne** désignent deux variables différentes.

Structure générale d'un programme

Nom de variables

Exemple:

- ❑ ageEtudiant, nom_etudiant, NOMBRE_Etudiants : **Noms valides.**
- ❑ Ageétudiant, nom etudiant: Noms non valides.

Structure générale d'un programme

Type de variables

Type de donnée	Signification	Taille (en octets)	Plage de valeurs acceptée
char	Caractère	1	-128 à 127
unsigned char	Caractère non signé	1	0 à 255
short int	Entier court	2	-32 768 à 32 767
unsigned short int	Entier court non signé	2	0 à 65 535
int	Entier	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned int	Entier non signé	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	0 à 65 535 0 à 4 294 967 295
long int	Entier long	4 (processeur 32 bits) 8 (processeur 64 bits)	-2 147 483 648 à 2 147 483 647 -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807
unsigned long int	Entier long non signé	4 (processeur 32 bits) 8 (processeur 64 bits)	0 à 4 294 967 295 0 à 18 446 744 073 509 551 615

Structure générale d'un programme

Type de variables

Type de Base	Taille du type en octets	Taille de l'exposant	Taille de la mantisse	Nombres de valeurs possibles
float	4	8 bits	23 bits	4 294 967 296
double	8	11 bits	52 bits	18 446 744 073 509 551 616
long double	12	15 bits	64 bits	79 008 162 513 705 374 134 343 950 336

Structure générale d'un programme

Type de variables

- ❑ Le type **bool** permet de représenter les valeurs **true** et **false**. C'est-à-dire 0 ou 1.
- ❑ Il ne s'agit d'un type pratique permettant de distinguer lors de la lecture du code une valeur qui est destinée à une utilisation logique (vraie ou fausse).
- ❑ En pratique, le type **bool** prend autant de place mémoire que le type **int**.

Structure générale d'un programme

Type de variables

- ❑ Un caractère (de type char) est un élément de la table **ASCII** codé sur un octet. Il s'agit en fait d'un nombre entre 0 et 127.
- ❑ Le caractère '**a**' est donc la valeur 97. '**A**' se code 65.
- ❑ **c = 'a'**, donne la valeur 97 à la variable c. Lorsqu'on affiche c. Ce n'est pas 97 qui apparaît, mais '**a**' car le compilateur sait qu'on veut afficher un caractère et non sa valeur, grâce à son type.
- ❑ **c = 'a' + 1** il s'agit bien d'un nombre, valide en C++, qui affiche le caractère suivant '**a**' dans la table '**b**'.

Structure générale d'un programme

Type de variables

Pour le type string:

- ❑ Le C et le C++ n'ont pas de type de base pour représenter les chaînes de caractères (comme en JAVA le type « string »). Les chaînes de caractères sont représentées par des tableaux de caractères. Il existe cependant dans la librairie standard de C++ une classe **std::string**, mais il ne s'agit pas d'un type de base.
- ❑ Un peu comme pour le char, il faut mettre la chaîne de caractères entre guillemets et non pas entre des apostrophes.

Structure générale d'un programme

Type de variables

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
int main() {
```

```
    string nomEtudiant(" Mostefa Amine");
```

```
    int ageEtudiant(20);
```

```
    float moyGen(12.43);
```

```
    double pi(3.14159);
```

```
    bool estAdmis(true);
```

```
    char lettre('a');
```

```
    return 0; }
```

Structure générale d'un programme

Déclaration de constantes

- ❑ Il existe des situations où, une fois qu'une variable prend une valeur de départ, on souhaite garantir qu'elle ne change plus par la suite. Dans ce cas, il faut précéder la déclaration de la variable par le mot réservé `const` :

`const type identificateur(valeur_initiale);`

- ❑ L'affectation de la valeur d'une variable constante cause une erreur à la compilation.

- ❑ Exemple:

`const char c ('A');` //exemple de constante caractère

`const int i (2017);` //exemple de constante entière

Structure générale d'un programme

Types dérivés

Il existe d'autres dérivés des types fondamentaux et des types définis par le programmeur, cette dérivation se fait à l'aide d'opérateurs de déclaration. On peut distinguer quatre types de dérivation :

- ☐ Pointeur
- ☐ Référence
- ☐ Tableau

Structure générale d'un programme

Types dérivés : Pointeurs

❑ Un pointeur est une variable qui contient l'adresse d'un objet en mémoire, l'objet pointé peut être référencé via le pointeur. On peut utiliser les pointeurs pour la manipulation des objets alloués dynamiquement, création et manipulation de structures chaînées, etc.

Exemple:

```
int* ptrInt, q; // attention ptrInt est un pointeur sur un entier et q est un entier  
double* ptrDouble, *ptr, **data; // ptrDouble et ptr sont des pointeurs sur  
                                //des doubles et data pointe deux fois sur un double  
char *text ; // text pointe sur un caractère  
const char* PCste = "this is a string"; //PCste pointe sur une chaîne constante
```

Structure générale d'un programme

Types dérivés : Références

- ❑ En C++, on peut coller plusieurs étiquettes à la même case mémoire (ou variable).
- ❑ On obtient alors un deuxième moyen d'y accéder.
- ❑ On parle parfois d'alias, mais le mot correct en C++ est référence.
- ❑ Pour déclarer une référence sur une variable, on utilise une esperluette (&).

Structure générale d'un programme

Exemple:

```
#include <iostream>
using namespace std;
```

```
Valeur de i: 5 ||  || Valeur de j: 5 ||  Valeur de k: 5
Valeur de i: 7 ||  || Valeur de j: 7 ||  Valeur de k: 7
```

```
int main() {
    int i(5);
    int & j(i); // la variable 'j' fait référence à 'i'. On peut utiliser 'j' ou 'i'
    int k=j ;
    cout<<" Valeur de i: "<<i<< " ||  || "<<" Valeur de j: "<<j<<" || "<<" Valeur de k:
"<< k << endl;
    j=j+2; k=i ;
    cout<<" Valeur de i: "<<i<< " ||  || "<<" Valeur de j: "<<j<<" || "<<" Valeur de k:
"<<k << endl; return 0;
}
```

Structure générale d'un programme

Types dérivés : Tableaux

❑ Un tableau est une collection d'objets du même type, chacun de ces objet est accédé par sa position. La dimension du tableau doit être connue en temps de compilation.

❑ Exemple:

```
const int numPorts = 200;
```

```
double portTable[numPorts];
```

```
int bufferSize;
```

```
char buffer[bufferSize]; // ERROR: the value of bufferSize is unknown
```

```
buffer[bufferSize]; // ERROR: the value of bufferSize is unknown
```

❑ Le tableau peut être initialisé lors de la définition :

```
int groupTable[3] = {134, 85, 29};
```

```
int userTable[] = {10, 74, 43, 45, 89}; // 5 positions
```

Structure générale d'un programme

Conversion de Type

Faite Ce type d'opération consiste à modifier la façon d'interpréter la séquence de bits contenue dans une variable

a. Conversion implicite

automatiquement par le compilateur (non sans warnings) lorsque plusieurs types de données sont impliqués dans une opération.

❑ Lors de l'affectation d'une valeur à un objet, le type de la valeur est modifié au type de l'objet

❖ **int count = 5.890; // conversion à int 5**

❑ Chaque paramètre passé à une fonction est modifié au type de l'argument espéré par la fonction

❖ **extern int add(int first, int second); count = add(6.41, 10); // conversion à int 6**

❑ Dans les expressions arithmétiques, le type de taille supérieure détermine le type de conversion

❖ **int count = 3; count + 5.8978; // conversion à double 3.0 + 5.8978**

Structure générale d'un programme

Conversion de Type

b. Conversion explicite ou casting

Cette conversion est demandée par le programmeur

❑ Exemple :

```
int result = 34;
```

```
result = result + static_cast<int>(10.890);
```

❑ Peut être aussi écrit

```
result = result + int(10.890);
```

❑ Permet d'utiliser des pointeurs génériques

```
char* name;
```

```
void* genericPointer;
```

```
name = (char*)genericPointer; // Conversion explicite
```

Structure générale d'un programme

Expressions

En combinant des noms de variables, des opérateurs, des parenthèses et des appels de fonctions on obtient des expressions.

Opérateurs multiplicatifs

Opération	Symbole	Exemple	
Multiplication	*	résultat = a * b;	Si a=5 et b=2 résultat = 10
Division	/	résultat = a / b;	Si a=5 et b=2 résultat = 2
Modulo	%	résultat = a % b;	Si a=5 et b=2 résultat = 1

- ❑ Le Modulo représente le reste de la division entière. Cet opérateur n'existe que pour les nombres entiers.
- ❑ La division est entière si les deux arguments sont entiers.

Structure générale d'un programme

Expressions

Opérateurs additifs

Opérateur	Signification	Arité	Associativité
+	Addition unaire	unaire	Non
-	Négation unaire	unaire	Non
+	Addition	binaire	Gauche à droite
-	Soustraction	binaire	Gauche à droite

❑ Il est clair que « l'addition » unaire ne fait rien pour les types simples (nombres entiers et réels), mais grâce à la surcharge des opérateurs, on peut donner un sens à cet opérateur sur des types (=classes) complexes

Structure générale d'un programme

Expressions

Opérateurs de décalage

Opérateur	Signification	Arité	Associativité
<<	Décalage à droite	Binaire	Gauche à droite
>>	Décalage à gauche	Binaire	Gauche à droite

❑ Ces opérateurs sont assez peu employés dans leur contexte « C » (décalage de bits), mais ils prennent une importance considérable en C++ dans la manipulation des flux d'entrées/sorties

❑ Exemple :

int b=100 ; // b=1100100 en code binaire

std ::cout<< " (b<<1) = " <<(b<<1) << std ::endl ;

std ::cout<< " (b<<2) = " <<(b<<2) << std ::endl ;

std ::cout<< " (b>>1) = " <<(b>>1) << std ::endl ;

std ::cout<< " (b>>2) = " <<(b>>2) << std ::endl ;

```
(b<<1) = 200
```

```
(b<<2) = 400
```

```
(b>>1) = 50
```

```
(b>>2) = 25
```

```
Process returned 0 (0x0)   execution time : 6.102 s
Press any key to continue.
```

Structure générale d'un programme

Expressions

Opérateurs relationnels et d'égalité

Opérateur	Signification	Arité	Associativité
<	Inférieur à	Binaire	Gauche à droite
>	Supérieur à	Binaire	Gauche à droite
<=	Inférieur ou égal à	Binaire	Gauche à droite
>=	Supérieur ou égal à	Binaire	Gauche à droite
==	Egalité	Binaire	Gauche à droite
!=	Inégalité	Binaire	Gauche à droite

- ❑ En C++, ces opérateurs retournent un type bool (valeurs true ou false) contrairement au C où ils retournent un type int (valeurs 0 ou 1)
- ❑ Erreur classique: ne pas confondre l'opérateur d'égalité (==) avec l'opérateur d'affectation (=).

Structure générale d'un programme

Expressions

Opérateurs sur les bits

Opérateur	Signification	Arité	Associativité
&	Et binaire	Binaire	Gauche à droite
^	Ou exclusif binaire	Binaire	Gauche à droite
	Ou binaire binaire	Binaire	Gauche à droite

❑ Ces opérateurs travaillent sur les bits, comme les décalages. Attention de ne pas les confondre avec les opérateurs logiques.

Structure générale d'un programme

Expressions

Opérateurs logiques

Opérateur	Signification	Arité	Associativité
&&	Et logique	Binaire	Gauche à droite
	Ou exclusif binaire	Binaire	Gauche à droite
e1 ?e2 :e3	Ou logique	ternaire	De droite à gauche

- ❑ Ces opérateurs travaillent sur le type bool et retournent un bool.
- ❑ Le dernier opérateur signifie: « si e1 est vraie alors faire e2, sinon faire e3 »

Structure générale d'un programme

Opérateur	Signification	Arité	Associativité
=	Affectation	$R = a$	Droite à gauche
*=	Multiplication et affectation	$a = a * b$	Droite à gauche
/=	Division et affectation	$a = a / b$	Droite à gauche
%=	Modulo et affectation	$a = a \% b$	Droite à gauche
+=	Addition et affectation	$a = a + b$	Droite à gauche
-=	Soustraction et affectation	$a = a - b$	Droite à gauche
<<=	Décalage à gauche et affectation	$a = a << b$	Droite à gauche
>>=	Décalage à droite et affectation	$a = a >> b$	Droite à gauche
&=	Et binaire et affectation	$a = a \& b$	Droite à gauche
=	Ou binaire	$a = a b$	Droite à gauche
^=	Ou exclusif binaire et affectation	$a = a \wedge b$	Droite à gauche
++	Incrémentation	$a = a + 1$	Droite à gauche
--	Décrémentement	$a = a - 1$	Droite à gauche

Structure générale d'un programme

Expressions

Opérateurs d'affectation

Exemple:

int a, b=3, c, d=3 ;

a = ++b ; // équivalent à b++ ; puis a=b ; => a=b=4

c = d++ ; // équivalent à c=d ; puis d++ ; => c=3 et d=4

Structure générale d'un programme

Les entrées/sorties en C++

❑ On peut utiliser les routines d'E/S de la bibliothèque standard de C (<stdio.h>). Mais C++ possède aussi ses propres possibilités d'E/S.

❑ Les nouvelles possibilités d'E/S de C++ sont réalisées par l'intermédiaire des opérateurs

❖ << (sortie), >> (entrée) et des flots (stream) définis dans la bibliothèque <iostream>, suivants

➤ **cin** : flot d'entrée correspondant à l'entrée standard (instance de la classe **istream_withassign**)

➤ **cout** : flot de sortie correspondant à la sortie standard (instance de la classe **ostream_withassign**)

➤ **cerr** : flot de sortie correspondant à la sortie standard d'erreur (instance de la classe **ostream_withassign**)

Structure générale d'un programme

Les entrées/sorties en C++

Syntaxes :

❑ **cout** << **exp_1** << **exp_2** << << **exp_n** ;

exp_k : expression de type de base ou chaîne de caractères

❑ **cin** >> **var_1** >> **var_2** >> >> **var_n** ;

var_k : variable de type de base ou char*

❑ Tous les caractères de formatage comme '**t**', '**n**' peuvent être utilisés.

Par ailleurs,

❑ l'expression **endl** permet le retour à la ligne et le vidage du tampon.

❑ On peut citer quelques avantages des nouvelles possibilités d'E/S :

- ❖ Vitesse d'exécution plus rapide.
- ❖ Il n'y plus de problème de types
- ❖ Autres avantages liés à la POO.

Structure générale d'un programme

Les entrées/sorties en C++

Exemple :

```
#include <iostream> // Standard C++ I/O
using namespace std;
int main() {
    int val1, val2;
    cout << "Entrer deux entiers: " << endl;
    cin >> val1 >> val2;
    cout << "Valeurs entrées: " << val1 << " et " << val2 << endl;
    cout << "valeur 1+valeur 2 =" << val1 + val2 << endl;
    return 0;
}
```

Structure générale d'un programme

Bibliothèque `io manip`

Il existe d'autres possibilités de modifier la façon dont les éléments sont lus ou écrits dans le flot:

- ❑ **`dec`**: Lecture/écriture d'un entier en décimal.
- ❑ **`oct`**: Lecture/écriture d'un entier en octal.
- ❑ **`hex`**: lecture/écriture d'un entier en hexadécimal.
- ❑ **`endl`**: Insère un saut de ligne et vide les tampons.
- ❑ **`setw(int n)`**: Affichage de `n` caractères.
- ❑ **`setprecision(int n)`**: Affichage de la valeur avec `n` chiffres avec éventuellement un arrondi de la valeur.
- ❑ **`setfill(char)`**: Définit le caractère de remplissage flush vide les tampons après écriture.

Structure conditionnelle « if »

La structure conditionnelle if permet de réaliser un test et d'exécuter une instruction ou non selon le résultat de ce

Syntaxe :

```
if (condition)  
{ instruction ; }
```

□ où condition est une expression dont la valeur est booléenne ou entière. Toute valeur non nulle est considérée comme vraie. Si le test est vrai, instruction est exécutée.

❖ Il est possible de définir plusieurs conditions à remplir avec les opérateurs ET et OU (&& et ||). Par exemple, l'instruction ci-dessous exécutera les instructions si l'une ou l'autre des deux conditions est vraie :

❖ Exemple:

```
if ((condition1) || (condition2))  
{ instruction ; }
```

Structure conditionnelle « if »

- ❑ L'instruction **if** dans sa forme basique ne permet de tester qu'une condition, or la plupart du temps on aimerait pouvoir choisir les instructions à exécuter en cas de non réalisation de la condition.
- ❑ L'expression **if ... else** permet d'exécuter une autre série d'instructions en cas de non-réalisation de la condition.

❑ Syntaxe :

if (condition)

{ instruction ; }

else

{ autre instruction ; }

❑ **else if** : effectuer un autre test

Il est possible de faire plusieurs tests à la suite. Pour faire tous ces tests un à un dans l'ordre, on va avoir recours à la condition **else if** qui signifie « **sinon si** ». Les tests vont être lus dans l'ordre jusqu'à ce que l'un d'entre eux soit vérifié.

Structure conditionnelle « if »

```
#include <iostream>
using namespace std;
int main() {
    int nombre;
    cout << "Entrez un nombre : ";
    cin >> nombre;
    if (nombre > 0) {
        cout << nombre << " Le nombre est positif." << endl;
    } else if (nombre < 0) {
        cout << nombre << " Le nombre est négatif." << endl;
    } else {
        cout << nombre << " Le nombre est nul." << endl;
    }
    return 0;
}
```

Branchement conditionnel switch

❑ Dans le cas où plusieurs instructions différentes doivent être exécutées selon la valeur d'une variable de type intégral, l'écriture des if successifs peut être relativement lourde. Le C++ fournit donc la structure de contrôle switch, qui permet de réaliser un branchement conditionnel.

Syntaxe :

```
switch (choix) {  
    case valeur1 :  
        instruction1; break;  
    case valeur2 :  
        instruction2; break;  
    case valeur3 :  
        instruction3; break;  
    default:  
        instructionParDéfaut;  
}
```

Branchement conditionnel switch

- ❑ La variable choix est évaluée en premier. Son type doit être entier. Selon le résultat de l'évaluation, l'exécution du programme se poursuit au cas de même valeur.
- ❑ Si aucun des cas ne correspond et si default est présent, l'exécution se poursuit après default.
- ❑ Si en revanche default n'est pas présent, on sort du switch. Pour forcer la sortie du **switch**, on doit utiliser le mot-clé **break**.

Branchement conditionnel switch

```
#include <iostream>
using namespace std;
int main()
{   int a;
    cout << "Tapez la valeur de a : ";
    cin >> a;
    switch(a ){
        case 1 :
            cout << "a vaut 1" << endl; break;
        case 2 :
        case 4 :
            cout << "a vaut 2 ou 4" << endl; break;
        case 3 :
        case 7 :
        case 8 :
            cout << "a vaut 3, 7 ou 8" << endl;
    }
    return 0;
}
```

Structures de contrôle itératives

- ❑ Les structures de contrôle itératives sont des structures qui permettent d'exécuter plusieurs fois la même série d'instructions jusqu'à ce qu'une condition ne soit plus réalisée.
- ❑ On appelle ces structures des boucles.
- ❑ Il existe 3 types de boucles à connaître :
 - ❖ la boucle **for**,
 - ❖ la boucle **while**
 - ❖ la boucle **do ... while**.

Structures de contrôle itératives

Boucle for

❑ La structure de contrôle for est sans doute l'une des plus importantes. Elle permet de condenser:

- ❖ **Un compteur**: une instruction (ou un bloc d'instructions) exécutée avant le premier parcours de la boucle du for. Il consiste à préciser le nom de la variable qui sert de compteur et éventuellement sa valeur de départ.
- ❖ **Une condition**: une expression dont la valeur déterminera la fin de la boucle.
- ❖ **Une itération**: une instruction qui incrémente ou décrémente le compteur.

Syntaxe:

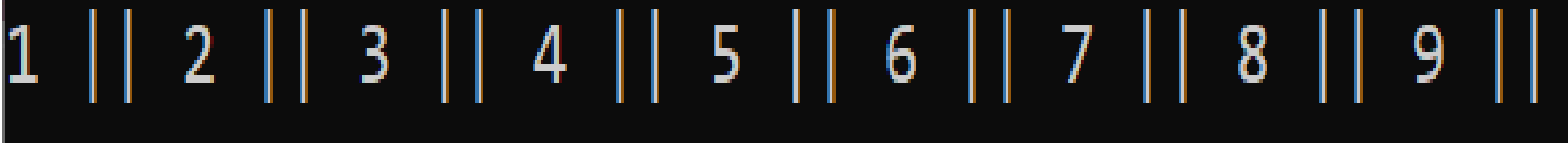
```
for (compteur; condition; itération) {  
    Instructions ;  
}
```

Structures de contrôle itératives

Boucle for

Exemple :

```
#include <iostream>
using namespace std;
int main() {
    int compteur(0);
    for (compteur = 1 ; compteur < 10 ; compteur++) {
        cout << compteur << " || ";
    }
    cout << endl ;
    return 0 ;
}
```



1 || 2 || 3 || 4 || 5 || 6 || 7 || 8 || 9 ||

Structures de contrôle itératives

Boucle while

❑ Le **while** permet d'exécuter des instructions en boucle tant qu'une condition est vraie.

❑ Syntaxe:

```
while (condition)
    { instructions ; }
```

❑ Exemple :

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
    while (i < 10) { //affiche les valeurs de 1 jusqu'à 9
        cout <<" "<< i <<endl ;
        i++ ;
    }
    return 0 ; }
```

Structures de contrôle itératives

Boucle while

❑ Le **while** permet d'exécuter des instructions en boucle tant qu'une condition est vraie.

❑ Syntaxe:

```
while (condition)
    { instructions ; }
```

❑ Exemple :

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
    while (i < 10) {    //affiche les valeurs de 1 jusqu'à 9
        cout <<" " << i <<endl ;
        i++ ;
    }
    return 0 ; }
```

1

2

3

4

5

6

7

8

9

Structures de contrôle itératives

Boucle **do ... while**

- ❑ La structure de contrôle **do ... while** permet, tout comme le **while**, de réaliser des boucles en attente d'une condition.
- ❑ Cependant, contrairement à celui-ci, le **do ... while** effectue le test sur la condition après l'exécution des instructions. Cela signifie que les instructions sont toujours exécutées au moins une fois, que le test soit vérifié ou non.
- ❑ **Syntaxe:**

```
do {  
    instructions ;  
} while (condition) ;
```

Structures de contrôle itératives

Boucle do ... while

Exemple:

```
#include <iostream>
using namespace std;
int main() {
    int i = 1;
    do {
        cout << i << endl;
        i++;
    } while(i<10);    //affiche les valeurs de 1 jusqu'à 9
    return 0;
}
```

Ruptures de Séquence

break

❑ L'instruction `break` sert à "casser" ou interrompre une boucle (**for**, **while** et **do ... while**), ou un **switch**. L'exécution reprend immédiatement après le bloc terminé.

❑ Exemple:

```
#include <iostream>
using namespace std;
int main() {
    int i;
    for (i = 0 ; i < 10 ; i++) {
        cout << " i= " << i << endl;
        if (i==3)
            break;
    }
    cout << " Valeur de i lors de la sortie de la boucle : " << i << endl ;
    return 0;
}
```

```
i= 0
i= 1
i= 2
i= 3
Valeur de i lors de la sortie de la boucle : 3
```

Ruptures de Séquence

continue

❑ L'instruction continue sert à "continuer" une boucle (**for**, **while** et **do ... while**) avec la prochaine itération.

❑ Exemple:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int i;
    for (i = 0 ; i < 10 ; i++) {
        if (i==3)
            continue;
        cout <<" i= " <<i<< endl;
    }
    cout<<" Valeur de i lors de la sortie de la boucle : " << i <<endl ;
    return 0;
}
```

```
i= 0
i= 1
i= 2
i= 4
i= 5
i= 6
i= 7
i= 8
i= 9
Valeur de i lors de la sortie de la boucle : 10
```

Blocs d'instructions

- ❑ En C++, les instructions peuvent être regroupées dans des blocs, indépendamment de toute structure de contrôle : il suffit d'entourer une séquence d'instructions avec les accolades.
- ❑ En C++, les blocs ont une grande autonomie : ils contiennent leurs propres variables, dont la déclaration et l'initialisation sont faites à l'intérieur même du bloc. Les variables qui sont déclarées dans un bloc sont les **variables locales** au bloc : on ne peut plus les utiliser dès que l'on quitte le bloc dans lequel elles sont définies.
- ❑ Les variables déclarées hors de tout bloc, y compris du main, sont des variables globales. Il ne faut jamais déclarer de **variables globales** car il est extrêmement difficile de suivre la valeur d'une variable globale, qui est accessible n'importe où dans le programme. La bonne pratique est de déclarer les variables au plus près de leur utilisation.

❑ Exemple

```
if (i != 0) {  
    int j(0);  
    ...  
    j = 2 * i;  
    ...  
}
```

// A partir d'ici, on ne peut plus utiliser j

Blocs d'instructions

Portée

- La **portée** d'une variable est l'ensemble des lignes de code où la variable est accessible. Par exemple, la portée d'une variable déclarée dans le corps de l'instruction if est le bloc contrôlé par l'instruction if: on ne peut plus l'utiliser dès que l'on quitte le bloc.

```
if (i != 0) {  
    int j(0);  
    ...  
    j = 2 * i;  
    ...  
    if (j != 2) {  
        int k(0);  
        ...  
        k = 3 * i;  
        ...  
    }  
    ...  
}
```

Blocs d'instructions

Règles de résolution de portée

```
if (i != 0) {  
    int j(0);  
    ...  
    j = 2 * i;  
    ...  
    if (j != 2) {  
        int j(0);  
        ...  
        j = 3 * i;  
        ...  
    }  
    ...  
}
```

- ❑ En cas d'ambiguïté, c'est-à-dire quand plusieurs variables de portées différentes portent le même nom:
 - ❖ la variable « la plus proche » est choisie
- ❑ Bonnes pratiques:
 - ❖ Évitez d'utiliser plusieurs fois le même nom de variables, sauf peut-être pour des variables locales aux boucles, comme i, j, ...

Blocs d'instructions

Portée : cas des itérations

❑ La déclaration d'une variable à l'intérieur d'une itération est une déclaration locale au bloc de la boucle, et aux deux instructions de test et d'incrément:

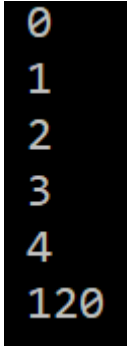
❑ Exemple :

```
for(int i(0); i < 5; ++i) {  
    cout << i << endl;  
}
```

// A partir d'ici, on ne peut plus utiliser ce i

❑ Exemple à ne pas suivre !

```
#include <iostream>  
using namespace std;  
int main(){  
    int i(120);  
    for(int i(0); i < 5; ++i) {  
        cout << i << endl;  
    }  
    cout << i << endl;  
    return 0;  
}
```



0
1
2
3
4
120